

Northwind Traders - Sales Analysis and Dashboard

Data Analytics and Visualization - Course Project Documentation

PostgreSQL (DBeaver) + Power BI Desktop

Project objective: build an end-to-end analytical solution that prepares relational sales data in SQL, models it as a Star Schema, calculates reusable business measures with DAX, and presents the results in an interactive Power BI report.

1. Project Overview

The project was implemented as a layered analytics solution. Each layer has a clear responsibility: SQL prepares reusable analytical datasets, Power Query applies only light presentation-oriented transformations, the Power BI model defines the analytical relationships, DAX contains business calculations, and report pages present the results to the user.

- Database layer: PostgreSQL / DBeaver for exploration, joins, filtering and reusable views.
- Transformation layer: Power Query for data types, readable column names and removal of connector navigation columns.
- Model layer: a Star Schema centred on `v_order_sales` with single-direction relationships.
- Calculation layer: a dedicated `_Measures` table for reusable DAX measures.
- Presentation layer: three interactive Power BI pages with slicers, reset filters and drill-through navigation.

Why this approach?

The solution deliberately pushes structural preparation into SQL instead of reproducing joins and business formulas in Power Query. This keeps Power BI focused on semantic modeling, DAX and visualization, makes the logic easier to validate, and avoids duplicating transformations across report pages.

2. Database Exploration and SQL Preparation

The Northwind tables were first explored to understand the grain of orders and order details, the available dimensions, the date range, active customers and discount behaviour. The analysis established validation baselines before Power BI was introduced.

Analytical views

- `v_Product_Enriched` combines products with categories and suppliers to create a clean product dimension.
- `v_Order_Sales` joins orders with order_details at order-line grain and calculates Total Line Amount directly in SQL as `unit_price * quantity * (1 - discount)`.
- `v_Customer_Geo` keeps only customers that actually have orders and exposes the geographic attributes needed by the map visual.

The fact view was intentionally kept at one row per order line. This preserves the lowest useful sales grain, supports product-level drill-through, and allows higher-level measures such as revenue and order count to be aggregated correctly.

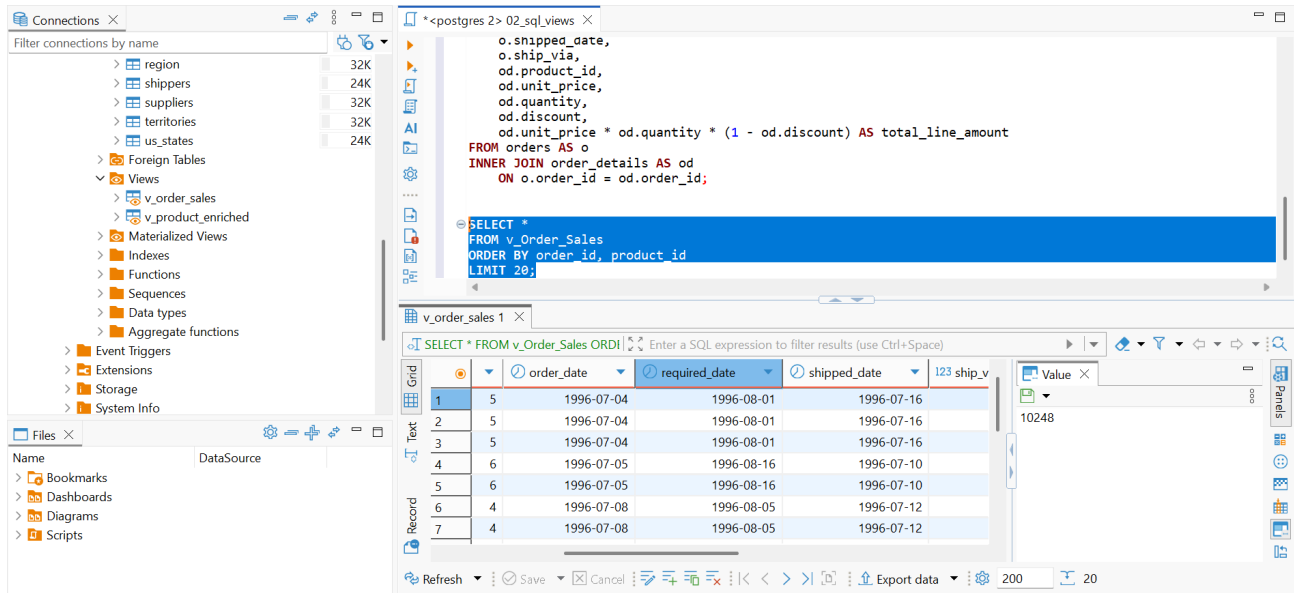


Figure 1. v_Order_Sales view: order-level attributes, line-level product data and Total Line Amount calculated in SQL.

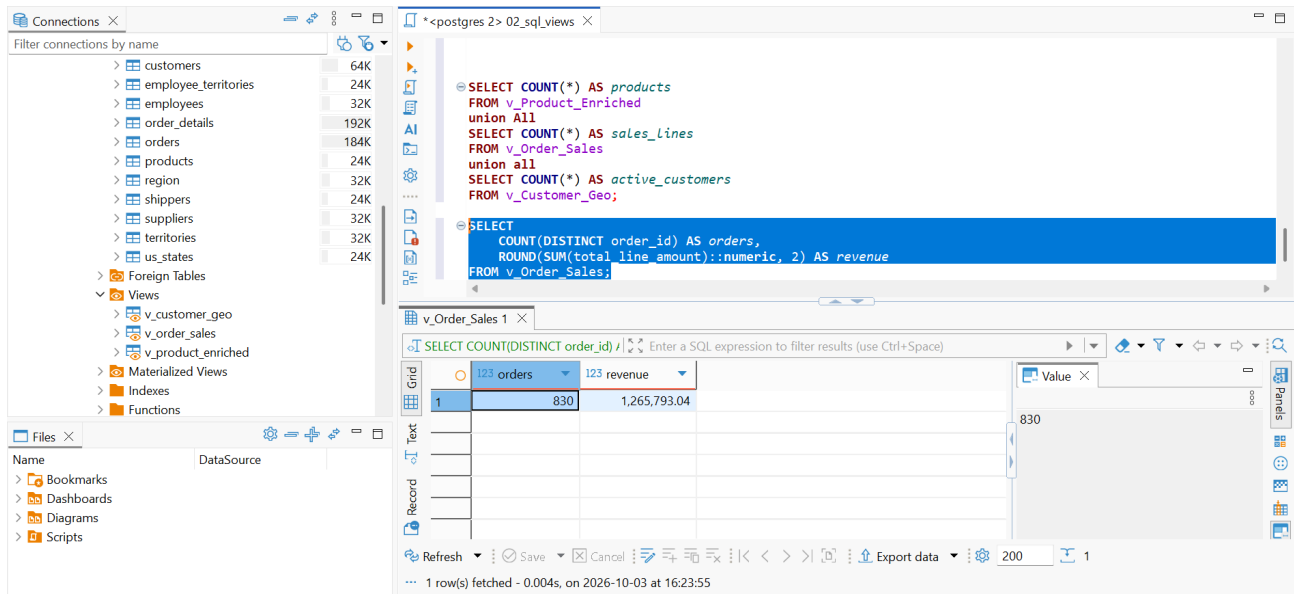


Figure 2. SQL validation: 830 unique orders and total revenue of 1,265,793.04.

3. Power Query Transformation

Power BI imported the three SQL views together with the employees and shippers tables. Power Query was used only for lightweight transformations: assigning the correct data types, converting technical field names into readable labels, and removing automatically generated navigation columns.

This limited use of Power Query was intentional. The heavier preparation had already been completed in SQL, so the report model remained simple and transparent.

Queries [5]

- v_product_enriched
- v_order_sales**
- v_customer_geo
- employees
- shippers

Table.RenameColumns("#changed Type",{{"order_id", "Order ID"}, {"customer_id", "Customer ID"}),

	i23 Order ID	AFC Customer ID	i23 Employee ID	Order Date	Required Date
1	10248 VINET		5	4.7.1996 p.	1.8.1996 p.
2	10248 VINET		5	4.7.1996 p.	1.8.1996 p.
3	10248 VINET		5	4.7.1996 p.	1.8.1996 p.
4	10249 TOMSP		6	5.7.1996 p.	16.8.1996 p.
5	10249 TOMSP		6	5.7.1996 p.	16.8.1996 p.
6	10250 HANAR		4	8.7.1996 p.	5.8.1996 p.
7	10250 HANAR		4	8.7.1996 p.	5.8.1996 p.
8	10250 HANAR		4	8.7.1996 p.	5.8.1996 p.
9	10251 VICTE		3	8.7.1996 p.	5.8.1996 p.
10	10251 VICTE		3	8.7.1996 p.	5.8.1996 p.
11	10251 VICTE		3	8.7.1996 p.	5.8.1996 p.
12	10252 SUPRD		4	9.7.1996 p.	6.8.1996 p.
13	10252 SUPRD		4	9.7.1996 p.	6.8.1996 p.
14	10252 SUPRD		4	9.7.1996 p.	6.8.1996 p.
15	10253 HANAR		3	10.7.1996 p.	24.7.1996 p.
16	10253 HANAR		3	10.7.1996 p.	24.7.1996 p.
17	10253 HANAR		3	10.7.1996 p.	24.7.1996 p.
18	10254 CHOPS		5	11.7.1996 p.	8.8.1996 p.
19	10254 CHOPS		5	11.7.1996 p.	8.8.1996 p.
20	10254 CHOPS		5	11.7.1996 p.	8.8.1996 p.
21					

12 COLUMNS, 999+ ROWS Column profiling based on top 1000 rows

Query Settings

PROPERTIES

Name
v_order_sales

All Properties

APPLIED STEPS

- Source
- Navigation
- Changed Type
- Rename Columns**

PREVIEW DOWNLOADED AT 16:33

Figure 3. `v_order_sales` after Power Query transformation: readable names and appropriate data types.

4. Data Model and Calendar

The Power BI semantic model uses a Star Schema. v_order_sales is the central fact table and is connected to Product, Customer, Employee, Shipper and Calendar dimensions. Relationships are active and use single-direction filtering from the one-side dimensions toward the many-side fact table.

A separate Calendar table was created in DAX from the minimum and maximum Order Date. It contains Year, Month Number, Month, Quarter and Year-Month. Month is sorted by Month Number, and Calendar is linked to v_order_sales through Order Date. This design enables reliable time-intelligence measures without introducing unnecessary bidirectional filters.

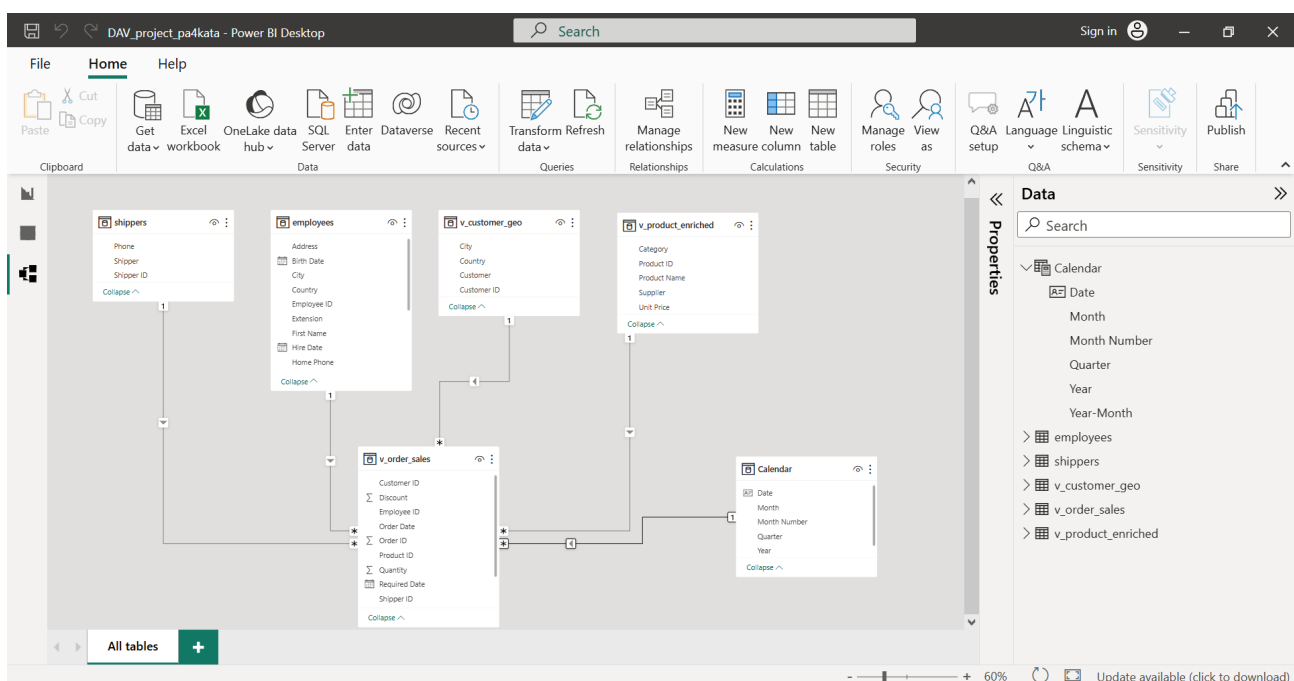


Figure 4. Final Star Schema with v order sales as the central fact table.

5. DAX Measures and Business Logic

All report measures were stored in a dedicated `_Measures` table. Reusable measures were preferred over calculated columns for aggregations, and VAR was used where it improved readability and filter-context handling.

- Basic measures: Total Revenue, Total Orders, Average Order Value and Total Quantity Sold.
- Time intelligence: Revenue YTD, Revenue Previous Year and YoY Growth %.
- Business logic: % Discounted and Top Product Sales (Top 3 products using TOPN).
- The YoY measure was adjusted so that a single-year selection compares that year with the same prior-year period, while the All context presents the latest visible year comparison instead of an ambiguous whole-dataset percentage.

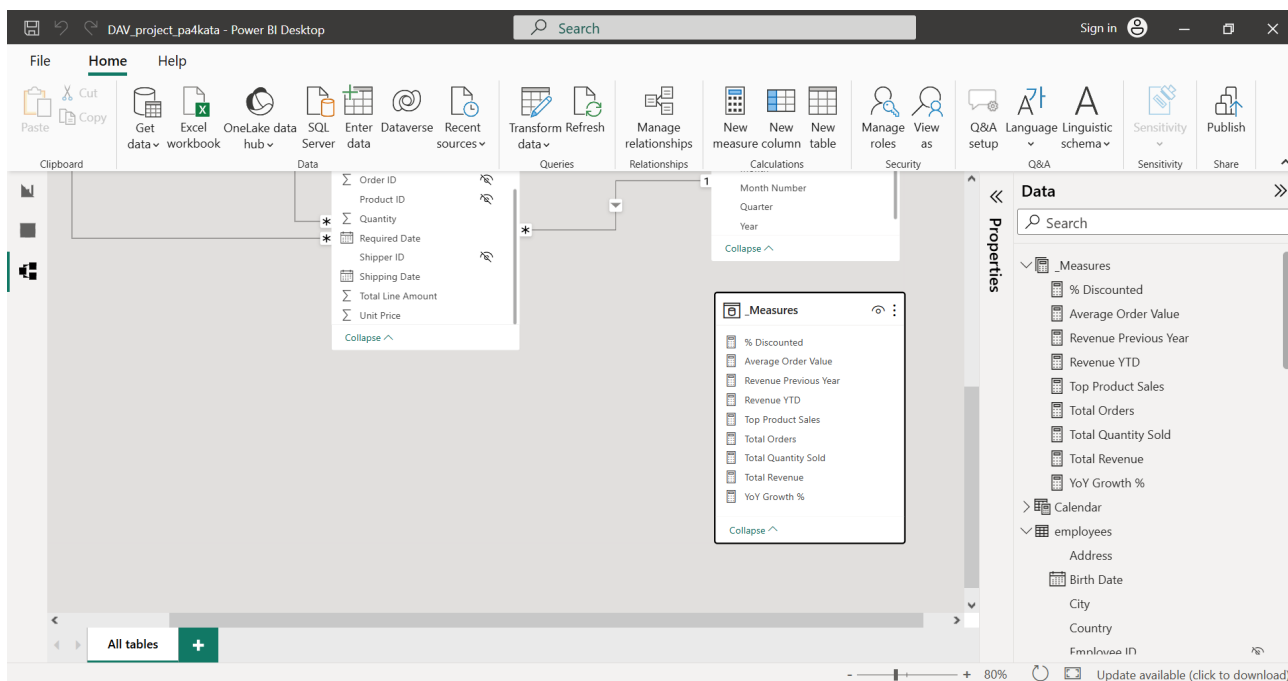


Figure 5. Dedicated `_Measures` table containing the nine required DAX measures.

Validation checkpoints

Metric	Validated value
Products	77
Sales lines	2,155
Active customers	89
Unique orders	830
Total Revenue	1,265,793.04
Total Quantity Sold	51,317
Average Order Value	1,525.05
% Discounted	40.69%
Top Product Sales (Top 3)	292,921.11

6. Dashboard Design

The report was divided into three pages so that users can move from high-level monitoring to detailed analysis and finally to transaction-level detail. This reduces visual clutter and gives each page a clear analytical purpose.

Page 1 - Executive Overview

The first page is designed for quick business monitoring. KPI cards summarize Total Revenue, YoY Growth, Total Orders and Average Order Value. A monthly trend compares current revenue with the previous year, a map shows geographic distribution, and the shipper chart shows order share. A Year slicer and Reset Filters bookmark support fast interactive exploration.

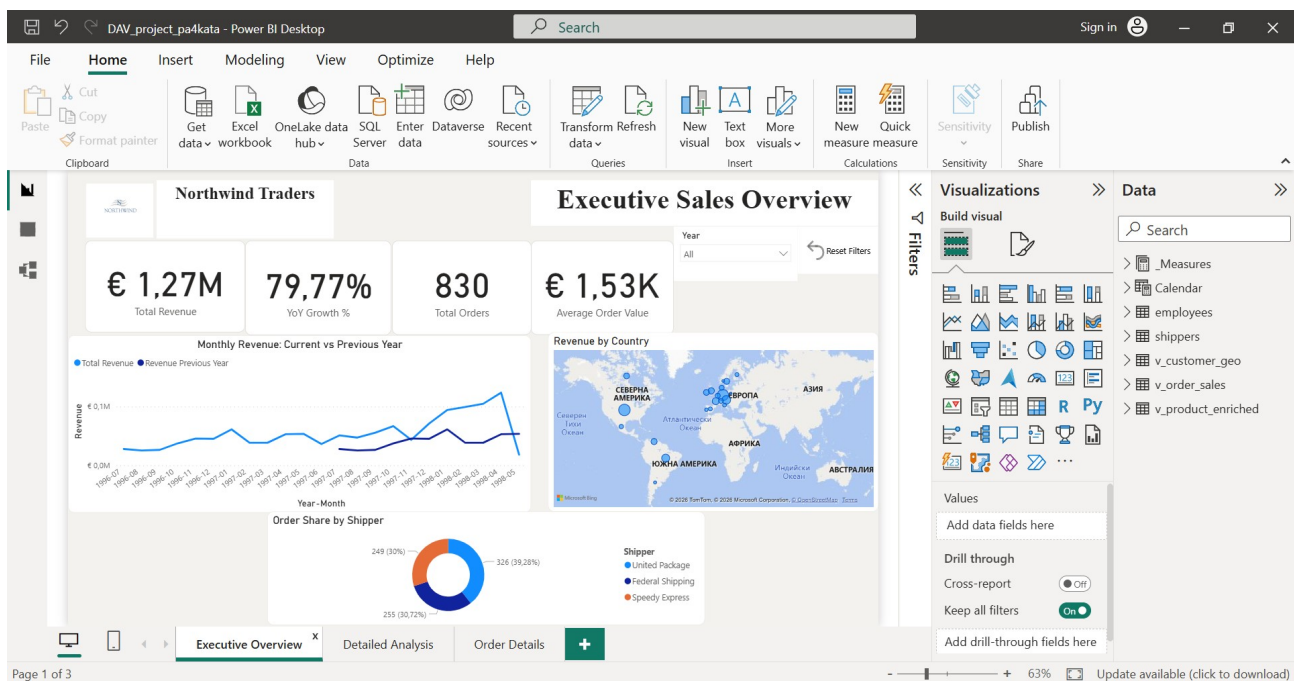


Figure 6. Executive Overview page.

Page 2 - Product & Employee Analysis

The second page moves from summary to diagnosis. It ranks the Top 10 Products by Revenue, compares employee sales performance and provides a Decomposition Tree that drills from Category to Product and Customer. Year, Category and Employee slicers allow the user to isolate a business segment, while Reset Filters restores the default state.

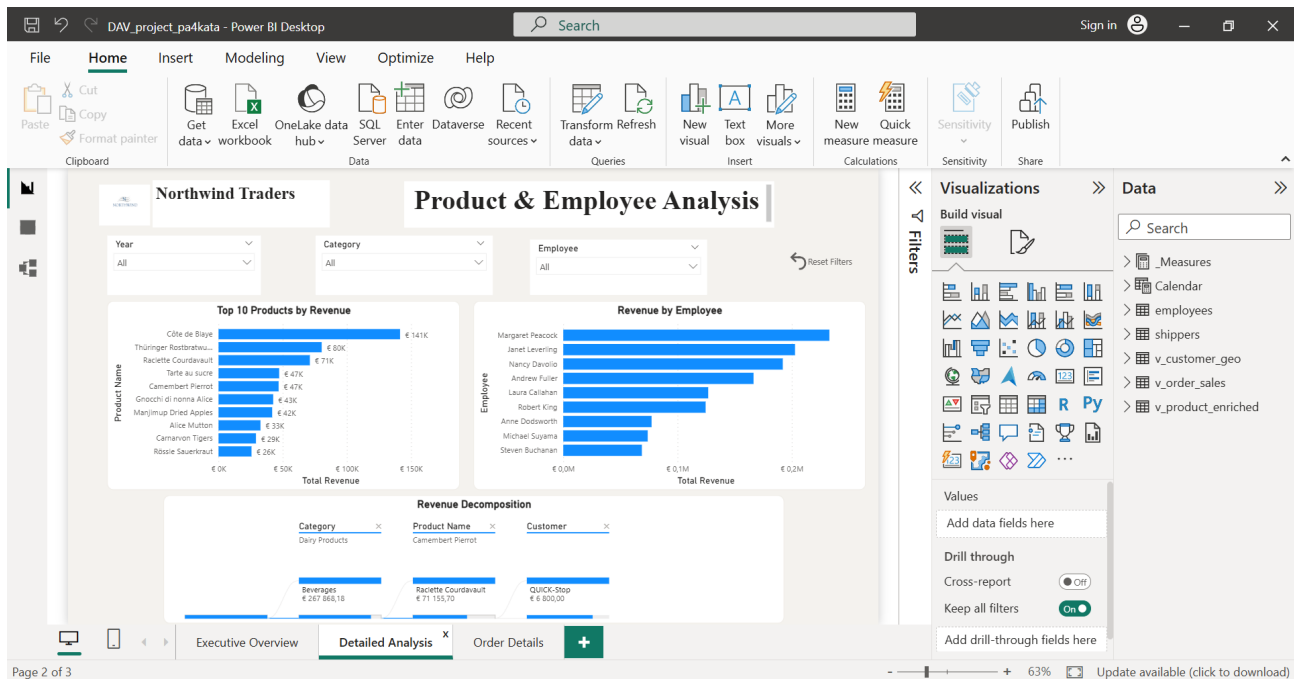


Figure 7. Detailed Analysis page with Top 10 Products, Employee performance and revenue decomposition.

Page 3 - Order Details Drill-through

The final page is a drill-through destination rather than a normal navigation page. Right-clicking a product on the analysis page passes Product Name as filter context to Order Details. The table then displays only the relevant orders, including Order ID, Order Date, Customer, Product, Quantity and Total Line Amount. A Back button returns the user to the previous analytical context.

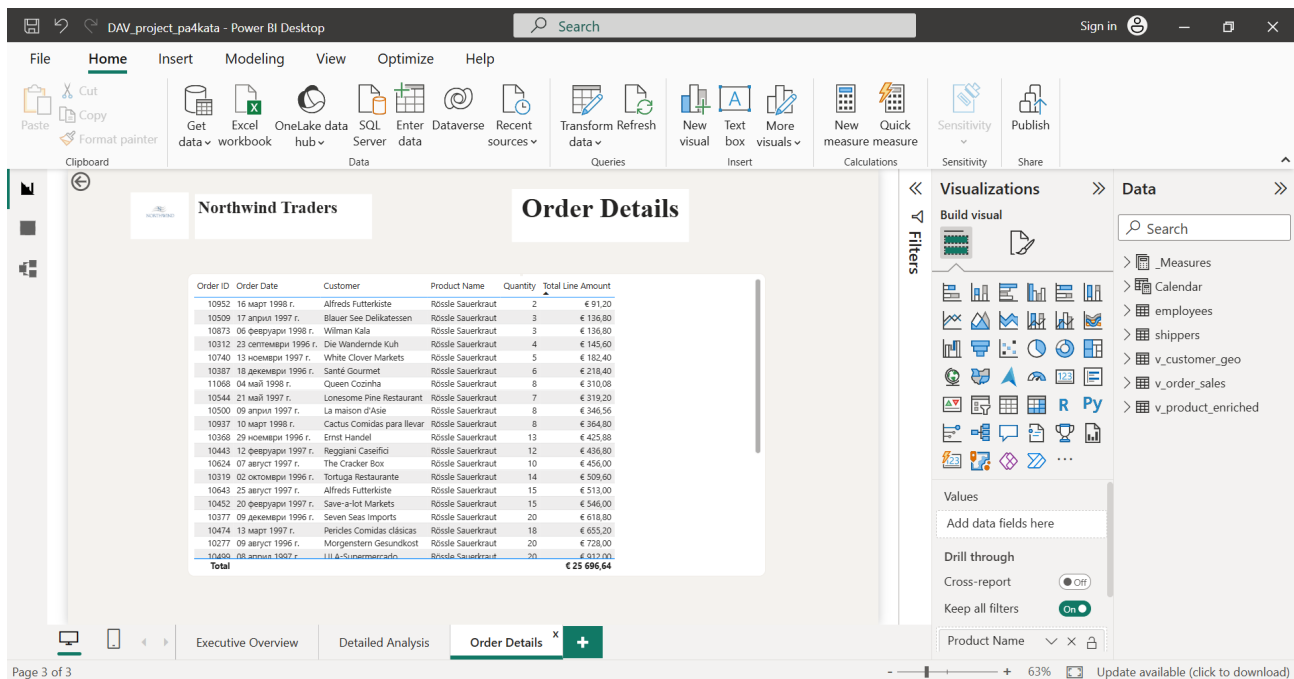


Figure 8. Product-level drill-through Order Details page.

7. Final QA and Interaction Testing

The report was tested end-to-end after the model and DAX calculations were completed. The checks focused on both numerical correctness and interactive behaviour.

- SQL totals and row counts were compared with Power BI measures.
- Year filtering was tested on the Executive Overview and the Reset Filters button was verified.
- Year, Category and Employee slicers were tested together on the Detailed Analysis page.
- The Decomposition Tree was expanded through Category -> Product -> Customer.
- Product drill-through was tested with multiple products, and the Back button returned correctly to the analysis page.
- The model was reviewed for accidental bidirectional relationships and unnecessary transformation logic.

8. Conclusion

The final solution provides a complete analytical workflow from relational source data to an interactive Power BI report. SQL performs reusable data preparation, the Star Schema provides a clear analytical model, DAX centralizes business calculations, and the report pages progressively expose summary, diagnostic and detailed information. The chosen architecture keeps the solution maintainable, easy to validate and aligned with standard BI modeling practices.